

Contents

ODE Protocol — Fish Length Estimation with Vision Foundation Models	2
1. Overview	2
1.1 Purpose and research question	2
1.2 Motivation	2
1.3 Problem definition	2
1.4 Entities and key concepts	3
1.5 Workflow summary	3
2. Data	3
2.1 Data source and provenance (completed)	3
2.2 Data description (completed)	3
2.3 Prediction target (completed)	3
2.4 Data splits and justification (completed)	4
2.5 Preprocessing (completed)	4
2.6 Augmentation (completed)	4
2.7 Known limitations and biases (completed)	4
3. Execution	5
3.1 Baseline model (completed)	5
3.2 VFM variant (planned)	5
3.3 Training setup (completed / in progress)	5
3.4 Evaluation (completed)	5
3.5 Experimental design (planned)	5
3.6 Results so far (completed)	6
3.7 Reproducibility (completed)	6
3.8 Next steps (planned)	7
References	7

ODE Protocol — Fish Length Estimation with Vision Foundation Models

Documentation following the ODE (Overview, Data, Execution) protocol for the standardized reporting of machine-learning workflows (Seuru et al., *Environmental Modelling & Software*, 2026).

Project	Fish length estimation: CNN baseline vs. Vision Foundation Model encoder
Course	Deep Learning for Maritime Vision Applications (Seminar)
Supervisor	Bohan Zhuang
Group members	Abu Bakar, M. Shahman Butt, Laksh Jiwani
Date	19 June 2026

Throughout this document we mark the status of each part as *completed*, *in progress*, or *planned*, so that it is clear what has already been carried out and what is still ahead.

1. Overview

1.1 Purpose and research question

The project looks at whether deep learning can estimate the length of individual fish directly from images in a reliable and scalable way, instead of relying on manual measurement, which is slow and labour-intensive in both commercial fisheries and scientific surveys.

The main question we want to answer is how well an established deep-learning baseline performs for fish length estimation, and whether replacing its encoder with a Vision Foundation Model (VFM) improves performance, especially in the case where labelled data are limited.

As an optional extension, we may later look at how well the approach transfers to a second fish dataset from the North Sea, if that dataset becomes available in time.

1.2 Motivation

Measuring the length of caught fish by hand is a bottleneck in fisheries work and in research surveys. A model that takes a segmented fish and predicts its length in centimetres would make this step automatic. The AutoFish benchmark already provides a CNN regression baseline based on MobileNetV2, and our aim is to test whether a modern self-supervised foundation model (DINOv2) used as the encoder gives better accuracy, and in particular whether it needs fewer labelled examples to reach the same quality. The limited-label setting is where pre-trained foundation features are expected to help the most, so it is the focus of the comparison.

1.3 Problem definition

This is a supervised regression problem with a single continuous output. The target we predict is the length of a fish in centimetres, taken from the per-instance `length` annotation, which is always a positive real number (roughly 14 to 45 cm across the species in AutoFish). The unit of prediction is one individual fish, defined by its segmentation mask, rather than a whole image, since each image contains several fish. The input is that single fish instance, given to the model as an RGB

image in which only the fish is visible, cropped to its bounding box, together with the normalized bounding-box coordinates.

1.4 Entities and key concepts

- Instance: one annotated fish, described by its mask, length, species, fish id and side.
- Group: a set of 60 images of the same physical batch of fish on the conveyor belt. The group is the unit we split on, so that the same fish never appears in both training and test data.
- Subsets within a group: images 1 to 20 (Set1) and 21 to 40 (Set2) show separated fish that do not touch, while images 41 to 60 (All) show fish that touch and occlude each other.
- Encoder: the image feature extractor, which is the part of the model we are studying and swapping.
- Regression head: the small MLP that maps the features (plus the bounding box) to a length.

1.5 Workflow summary

The pipeline goes from the COCO annotation to a mask, then to an RGB-masked square crop, a resize to 224 by 224, the encoder, a concatenation with the normalized bounding box, and finally the MLP head that outputs the length in centimetres. We train and evaluate two encoder variants, the MobileNetV2 baseline and the DINOv2 foundation model, using the same pipeline and the same data budgets so that the encoder is the only thing that changes.

2. Data

2.1 Data source and provenance (completed)

We use the AutoFish dataset (Bengtson et al., WACVW 2025), available at <https://huggingface.co/datasets/v>. Our local copy is in `data/autofish/` and contains the 25 groups of images together with a single COCO `annotations.json` file. The dataset is a public research benchmark and is used here as the primary basis for development and evaluation, as required by the task description.

2.2 Data description (completed)

The dataset has 1,500 RGB images at a resolution of 2464 by 2056, taken top-down over a conveyor belt with checkerboard calibration targets visible in the scene. There are 18,158 annotated fish instances in MS-COCO format. Each instance carries a bounding box, a segmentation mask, a category id, the `length` field in centimetres (our target), a fish id, the side that faces up, the area, the image id and the group number. Seven species are present: cod, haddock, hake, horse mackerel, whiting, saithe, and a catch-all “other” class. Each of the 25 groups consists of 60 images (20 in Set1, 20 in Set2 and 20 in All), and every fish appears 40 times per group, 20 times from each side, which gives repeated views at different levels of occlusion difficulty.

2.3 Prediction target (completed)

The regression target is the `length` field in centimetres attached to each instance. This is the physical length of the fish and is the same across all the repeated views of a given fish id within a group.

2.4 Data splits and justification (completed)

We split by group, following the AutoFish benchmark, because the same physical fish recurs across the 60 images of its group and a random per-image split would leak fish between training and test.

Split	Groups	Purpose
Train	2, 3, 4, 5, 7, 8, 9, 12, 13, 15, 16, 18, 19, 23, 24 (15 groups)	fitting the model
Validation	1, 6, 11, 17, 25 (5 groups)	early stopping and checkpoint selection
Test	10, 14, 20, 21, 22 (5 groups)	final evaluation, never used in training

We keep the same test split as the paper so that our numbers can be compared directly. After loading, this gives 10,760 training, 3,639 validation and 3,759 test instances.

2.5 Preprocessing (completed)

For each fish instance we apply the following steps in a fixed order. We decode the COCO segmentation into a binary mask, then apply RGB masking so that every background pixel is set to zero and only the fish remains. We crop to the bounding box and pad the shorter side so that the crop is square, which keeps the aspect ratio. We then resize to 224 by 224 and convert to a float tensor in the range 0 to 1. Finally, the bounding box is read back from the mask and normalized by the image dimensions (2464 and 2056), giving four values that are passed to the head alongside the image features.

These choices follow the baseline paper. The RGB masking removes the conveyor belt and other background that would otherwise confuse the model, the square padding avoids stretching the fish, and the normalized bounding box gives back the absolute-scale information that is lost when we crop, which the model needs to turn appearance into centimetres. Keeping the preprocessing identical to the paper means the encoder is the only variable we change.

2.6 Augmentation (completed)

During training only, we apply colour jitter (brightness 0.2, contrast 0.5, saturation 0.4, hue 0.3) to cope with the uncontrolled natural lighting noted in the dataset. We do not use any geometric augmentation such as flips or rotation, because that would distort the relationship between pixels and real length. The DINOv2 variant will additionally apply ImageNet mean and standard-deviation normalization, which the pre-trained encoder expects (planned).

2.7 Known limitations and biases (completed)

There is a strong class imbalance in both length and species; for example the test set has only 80 saithe but 1,000 haddock. Occlusion in the All subset can cut off part of a mask, which tends to make the predicted length too short. Length is also correlated with species, so the model could lean on species cues rather than pure geometry, something we look at in the analysis described in Section 3.5.

3. Execution

3.1 Baseline model (completed)

The baseline is the REG model from the AutoFish paper. The encoder is MobileNetV2 pre-trained on ImageNet, with its classification head removed so that it produces a 1280-dimensional feature vector. This vector is concatenated with the four bounding-box values and passed through an MLP head: Linear to 1000, batch norm, ReLU, Linear to 500, batch norm, ReLU, and Linear to 1. The model is trained with the L1 (mean absolute error) loss and the Adam optimizer at a learning rate of 0.001. The encoder is fully isolated in `length_estimation/cnn/Model.py`.

3.2 VFM variant (planned)

For the foundation-model variant we replace MobileNetV2 with DINOv2, loaded through `torch.hub`, either the ViT-S/14 version with 384 dimensions or the ViT-B/14 version with 768 dimensions. The MLP head stays the same, with its input width set to match the chosen embedding size. Our main configuration keeps the encoder frozen and trains only the MLP head, since this is the setting in which foundation-model features are expected to do well with few labels; a fully fine-tuned version will be run as a secondary comparison. The crops, the bounding-box input, the loss and the optimizer are all kept the same, and the encoder is selected through a `MODEL_BACKEND` option in the config, so the baseline and the VFM differ only in the encoder.

3.3 Training setup (completed / in progress)

All training so far has been done locally on an Apple M3 Pro laptop using PyTorch 2.12 on the MPS backend, with Python 3.12 in a uv-managed virtual environment. The original release was written for CUDA only, so we made the device selection automatic (CUDA, then MPS, then CPU). The reference protocol in the paper is 200 epochs at batch size 32. The run we report here used 10 epochs at batch size 16, which is the reduced configuration shipped in `default.cfg`; it took about 32 minutes and reached a best validation L1 of 2.91 cm. We fix the random seed to 42 and save both the last-epoch and the best-validation checkpoints for each run.

3.4 Evaluation (completed)

We evaluate with mean absolute error (MAE, in centimetres) and mean absolute percentage error (MAPE, in percent), the same metrics as the paper, reported overall, per subset (separated and touching) and per species. The main evaluation uses the ground-truth masks, which isolates the length estimator from the segmentation step; evaluating on predicted masks from Mask2Former for the full end-to-end error is planned. Evaluation is done only on the test groups 10, 14, 20, 21 and 22.

3.5 Experimental design (planned)

1. Headline comparison: baseline against the DINOv2 variant, trained on the full training split with the same budget, reported as MAE and MAPE on the test set.
2. Limited-label study, which is the core of the project: retrain both encoders on progressively smaller training subsets (for example 1, 3, 5, 9 and 15 groups), repeat each with several random samplings, and plot MAE against the number of training groups to compare how the two encoders degrade as labels are reduced.

3. Failure and bias analysis: error distributions, bias by species and by length range, the split between separated and touching (occluded) fish, and a look at the worst qualitative cases.
4. Optional transfer to a second North Sea dataset if it is released in time.

3.6 Results so far (completed)

The table below shows our reproduced REG baseline on the test split, using the ground-truth masks, next to the values reported in the AutoFish paper.

Subset	Our MAE	Our MAPE	Paper REG (gt) MAE	Paper MAPE
Separated	3.03 cm	9.18 %	0.67 cm	2.10 %
Touching	3.03 cm	9.17 %	0.96 cm	3.08 %
Combined	3.03 cm	9.18 %	0.82 cm	2.59 %

Per-species MAE in centimetres: horse mackerel 1.86, haddock 2.48, whiting 2.84, cod 2.87, other 3.07, saithe 5.66, hake 5.89.

The pipeline reproduces the baseline correctly. The data loading, the masking and cropping, the MobileNetV2 encoder with the bounding-box input and the MLP head, the L1 and Adam training loop, the checkpointing and the MAE and MAPE evaluation all run on the exact paper splits and give sensible results that improve steadily during training. The difference between our numbers and the published ones comes from how long the model was trained, not from the method.

The AutoFish authors train the regression model for 200 epochs at batch size 32 (their Section 4.2.2). Our reported run used 10 epochs at batch size 16, which is the reduced configuration the authors themselves provide in `default.cfg` for quick checks. At 10 epochs the validation loss is still falling: our best validation L1 is about 2.91 cm and it is still moving around 3 cm from epoch to epoch, so the model is under-trained rather than at its limit. Because the data, the architecture, the loss, the optimizer and the splits are all identical to the paper and only the number of epochs is different, we expect the full 200-epoch schedule to bring the model down to the paper’s combined MAE of about 0.82 cm, along with the matching per-subset and per-species values.

We ran the shorter schedule first for two practical reasons. The first is a hardware limitation. All of our training was done on an Apple M3 Pro laptop through the MPS backend, rather than on the NVIDIA data-centre GPUs the original work assumes. One epoch over the full training split takes roughly three minutes on this machine, so a complete 200-epoch run is an unattended job of about seven to eleven hours, whereas a 10-epoch pass finishes in around half an hour and lets us check the whole pipeline in a single sitting. The second reason is that our priority at this stage was to reproduce the baseline method correctly before committing that much GPU time, in other words to confirm that the full pipeline behaves as intended and improves as expected on the paper’s splits. With that confirmed, the immediate next step is to run the authors’ full 200-epoch schedule to reproduce the exact paper numbers, after which we will reuse the same converged setup for the Vision Foundation Model comparison so that both encoders are trained under identical, fully converged budgets.

3.7 Reproducibility (completed)

The original release sits in `autofish_training_release/`. Our changes to it are small and limited to making the device selection automatic, fixing a divide-by-zero in the epoch-timing

print, adding the local config files (`configs/baseline.cfg` and `configs/smoke.cfg`), and adding `compute_metrics.py` to compute MAE and MAPE. The environment is a virtual environment with Python 3.12, torch 2.12, torchvision, pycocotools, scikit-image, opencv-python, stocaching and pyransac3d. Runs are made deterministic through a fixed seed (42), the fixed group splits and the deterministic preprocessing. Training is started with `python cnn/train.py --config configs/baseline.cfg`, and evaluation with `python eval_length_estimators.py --gt_path ../annotations.json --cnn_model_path ../model.pt`.

3.8 Next steps (planned)

1. Run the full 200-epoch baseline to match the paper numbers.
2. Implement and train the DINOv2 variant, starting with the frozen-encoder configuration.
3. Carry out the limited-label study and produce the comparison curves.
4. Do the failure-case and bias analysis and write it up.
5. If the second dataset becomes available, run the optional transfer experiment.

References

- S. H. Bengtson, D. Lehotský, V. Ismiroglou, N. Madsen, T. B. Moeslund, and M. Pedersen, “AutoFish: Dataset and Benchmark for Fine-Grained Analysis of Fish,” 2025 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW), Tucson, AZ, USA, 2025, pp. 1513–1522.
- M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” Transactions on Machine Learning Research, 2024.
- S. Seuru, V. Grimm, M. Barton, L. Perez, N. Mahdizadeh Gharakhanlou, R. Sengupta, and A. M. Dagnino, “The ODE (Overview, Data, and Execution) protocol for a standardized use of machine learning in environmental, social and related interdisciplinary sciences,” Environmental Modelling & Software, 2026.